

DESARROLLO DE VIDEOJUEGOS

LABORATORIO 08

Docente: Marco Antonio Camacho Alatrasta

Alumno: Kenny Luis Flores Chacón

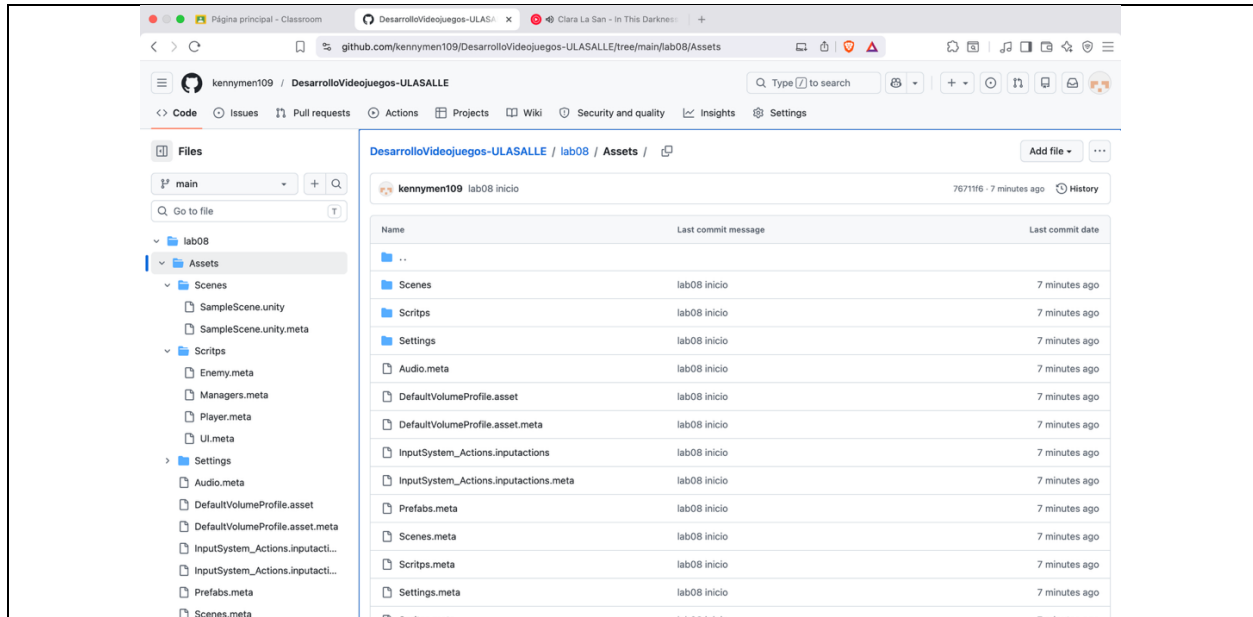
Repositorio

<https://github.com/kennymen109/DesarrolloVideojuegos-ULASALLE/tree/main/lab08>

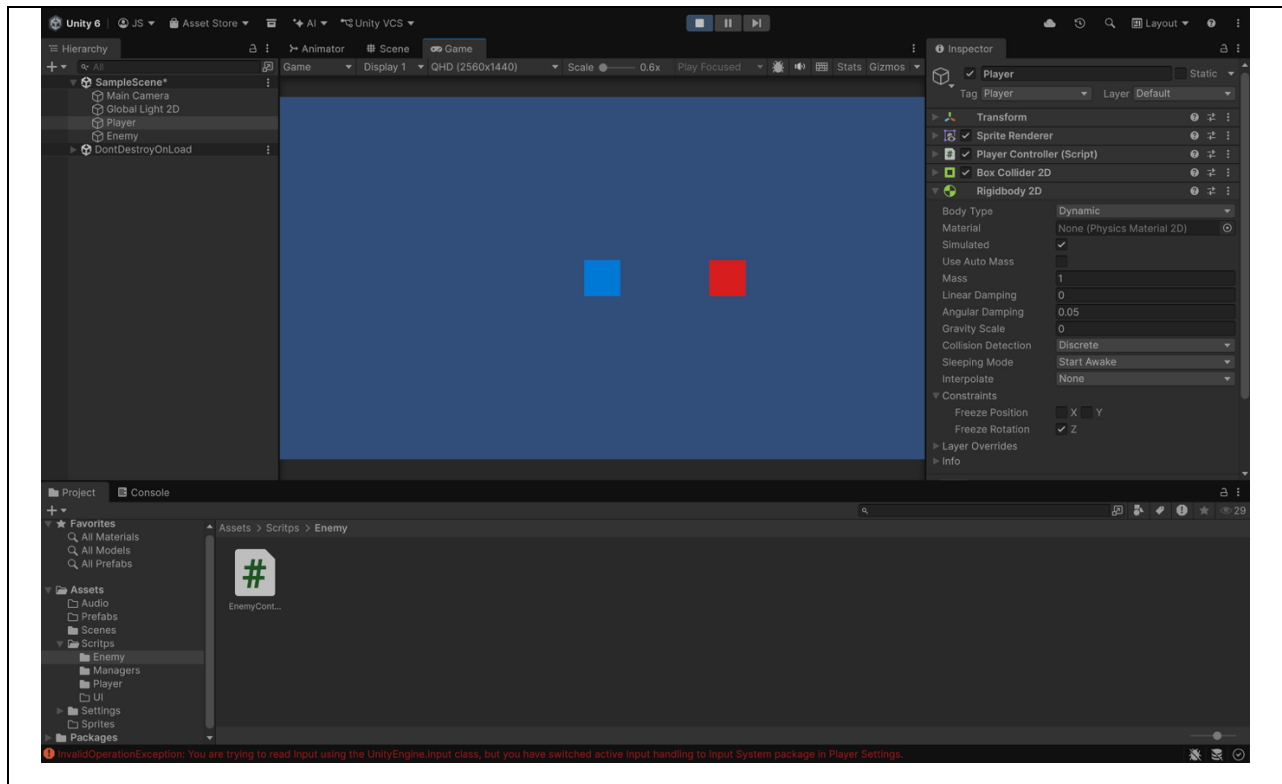


CAPTURAS DE PANTALLA

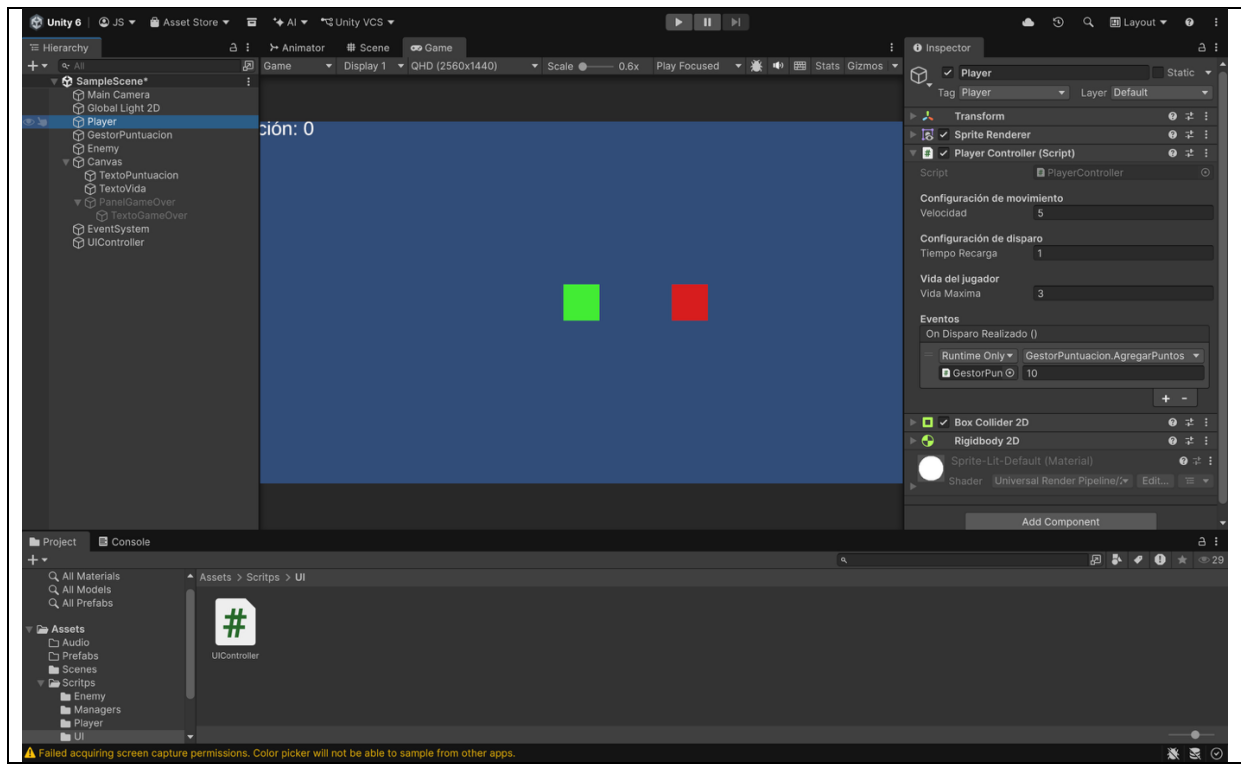
1. Captura 1



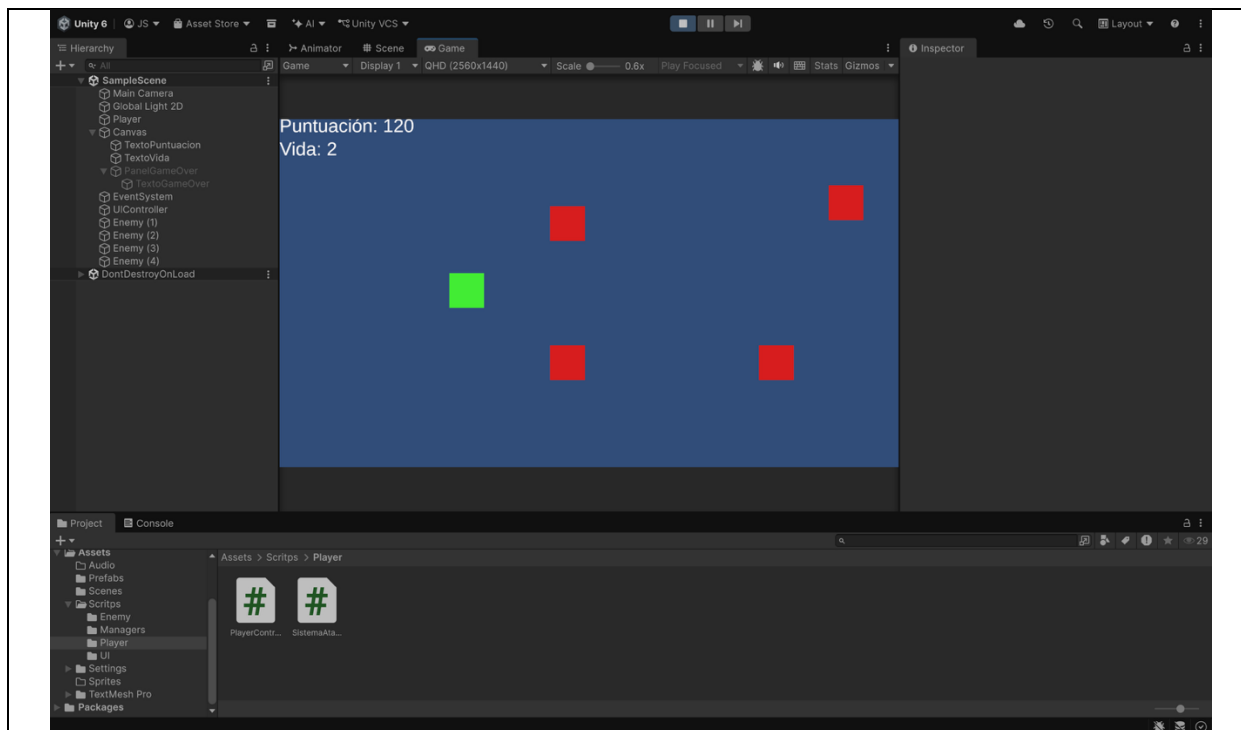
2. Captura 2



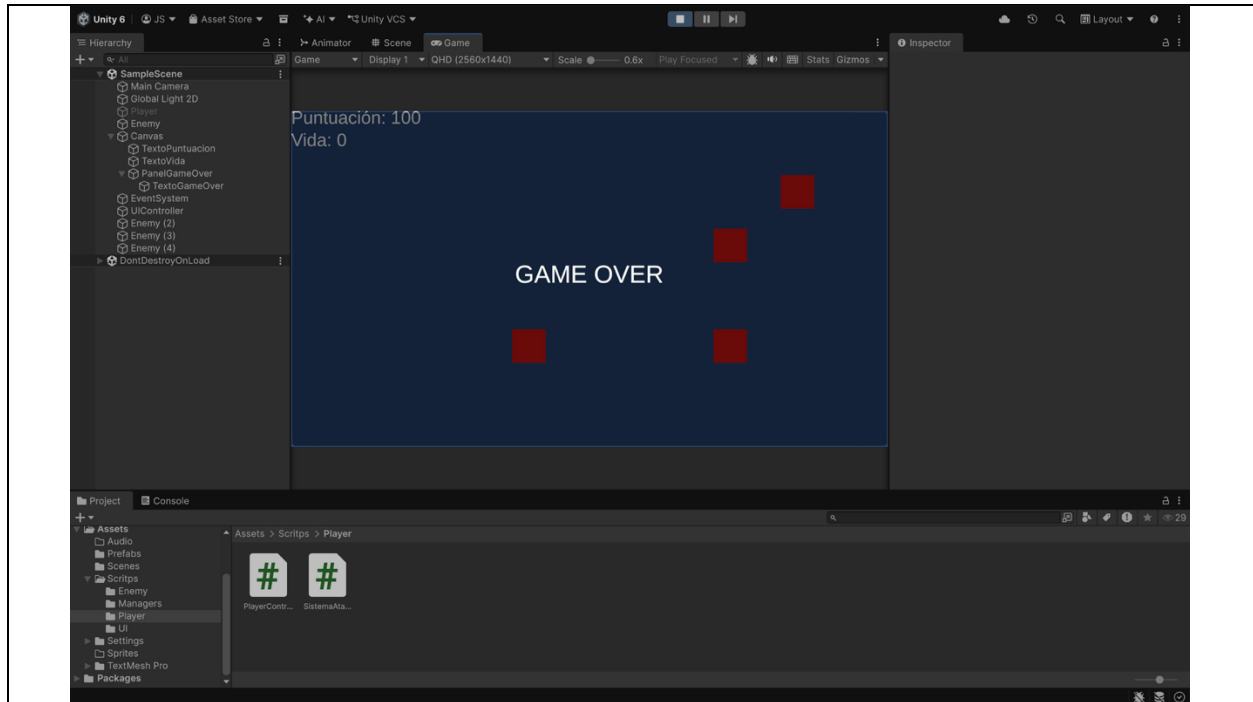
3. Captura 3



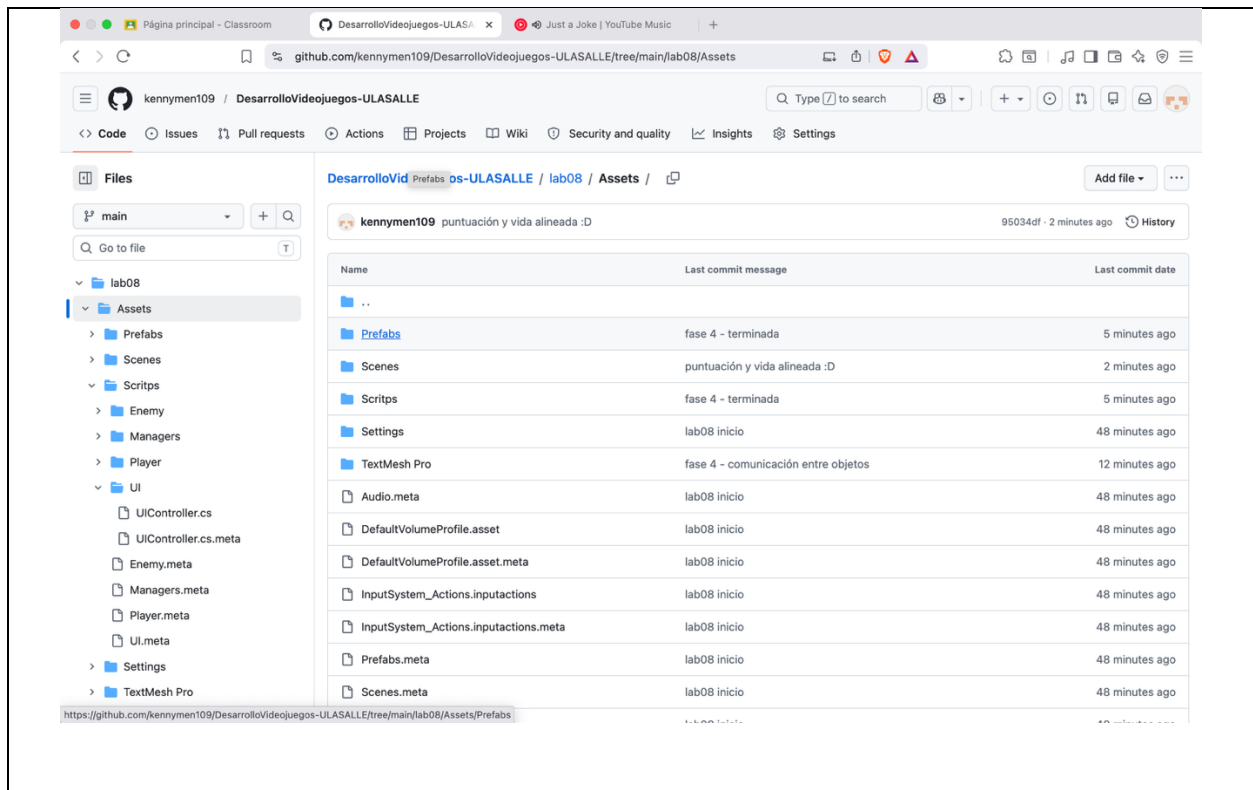
4. Captura 4

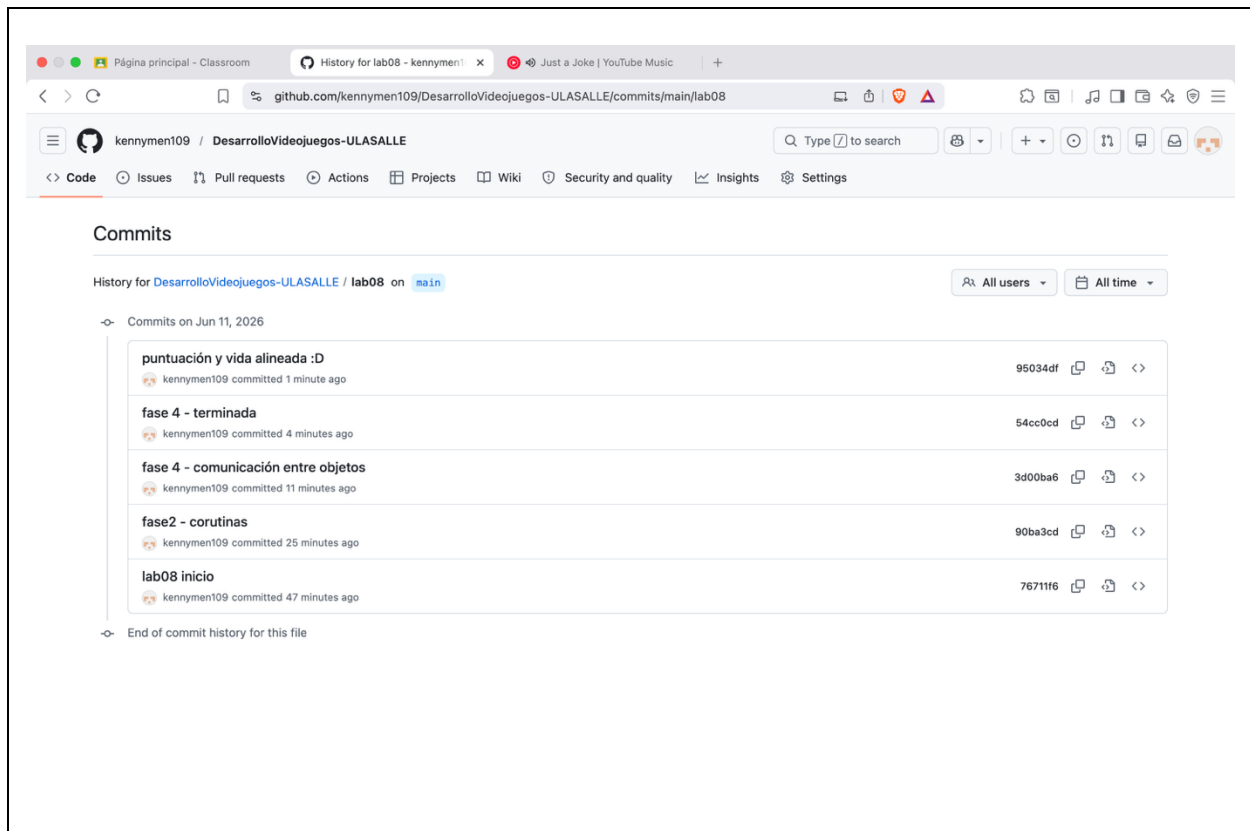


5. Captura 5



6. Captura 6.1 y 6.2





Preguntas de Reflexión

1.

Si usamos DontDestroyOnLoad, el objeto sigue existiendo aunque cambiemos de escena. El problema es que cuando volvemos a cargar la escena, Unity crea otro GestorPuntuacion nuevo. Entonces tendríamos dos objetos haciendo el mismo trabajo al mismo tiempo. Para evitar eso, el método Awake revisa si ya existe una instancia del gestor. Si encuentra una, el nuevo objeto se destruye automáticamente. De esta manera solo queda un GestorPuntuacion funcionando durante todo el juego y no hay problemas con la puntuación.

2.

Aunque haya un while (true), el juego no se congela porque las corrutinas usan yield return. Cuando la corrutina encuentra un yield, se detiene por un momento y le devuelve el control a Unity para que siga actualizando el juego y mostrando los siguientes frames.

Si no existiera el yield return, el ciclo se ejecutaría sin detenerse y Unity se quedaría congelado porque no podría realizar otras tareas.

3.

OnEnable se ejecuta cuando un objeto se activa y OnDisable cuando se desactiva. Si agregamos una suscripción en OnEnable pero la quitamos recién en OnDestroy, pueden aparecer problemas. Por ejemplo, si el objeto se desactiva y luego se vuelve a activar, podría registrarse varias veces al mismo evento.

Por eso es mejor suscribirse en OnEnable y cancelar la suscripción en OnDisable. Así se evita que las acciones se repitan más de una vez y el programa funciona correctamente.

Preguntas Finales

Pregunta Final 1:

Los dos sirven para avisar cuando ocurre algo dentro del juego, pero funcionan de manera diferente.

UnityEvent permite conectar acciones desde el Inspector de Unity sin necesidad de escribir más código. Es útil porque facilita la configuración de eventos.

System.Action funciona mediante código de C#. Se utiliza para que diferentes partes del juego puedan comunicarse entre sí, como el sistema de puntuación y la interfaz de usuario.

Pregunta Final 2:

Si la interfaz revisara la información constantemente usando Update, aparecerían dos problemas importantes.

Primero, la interfaz dependería demasiado de otros objetos del juego, haciendo el código más difícil de mantener. Segundo, se gastarían más recursos porque estaría comprobando datos en cada frame aunque no hayan cambiado. Con los eventos, la interfaz solo se actualiza cuando ocurre un cambio real, por lo que el juego funciona de una manera más eficiente.

Pregunta Final 3:

WaitForSeconds espera una cantidad fija de tiempo antes de continuar. En cambio, WaitUntil espera hasta que se cumpla una condición específica. Por ejemplo, en una pantalla de Game Over se puede esperar hasta que el jugador presione la tecla R para reiniciar la partida. Otro ejemplo es un enemigo que espera hasta que el jugador se acerque lo suficiente antes de comenzar a perseguirlo.

De esta forma, el juego responde a lo que ocurre en cada momento en lugar de depender únicamente del tiempo.